

## 데이터 모델링: 업무의 합의에서 기술적 통제까지

온라인 쇼핑몰의 주문 처리 과정을 데이터베이스, 즉 정보를 체계적으로 저장해 두는 저장소에 옮기는 작업은 현업 담당자와 개발자, 데이터베이스 관리자(DBA)가 같은 업무를 서로 다르게 이해한다는 문제에서 출발한다. 현업 담당자는 업무의 절차와 규칙을 정하고, 개발자는 이를 화면과 프로그램으로 구현하며, DBA는 데이터를 어디에 어떤 규칙으로 저장할지 정한다. 세 사람은 같은 단어도 다른 뜻으로 쓸 수 있다. 현업 담당자는 상품을 산 적 있는 사람을 모두 고객으로 보지만, 개발자는 회원가입을 마친 사람만 고객으로 처리하며, DBA는 비회원 주문자를 고객 정보와 함께 저장할지 따로 저장할지 결정해야 한다. 이 차이를 맞추지 않으면 하나의 요구사항이 화면마다 다르게 구현되고 데이터도 서로 다른 구조로 저장된다. 따라서 세 사람이 함께 보고 같은 뜻으로 읽을 수 있는 공통의 데이터 구조가 필요하다.

이 구조는 개체 관계 다이어그램, 즉 ERD로 표현되며 관계자들은 이 그림을 보면서 무엇을 관리할지부터 합의한다. 고객처럼 계속 등록하고 조회하고 수정해야 하는 대상을 엔터티라 하며, 고객·상품·주문·결제가 그 예다. 반면 고객의 이름처럼 다른 대상을 설명하는 정보이거나 “쇼핑몰은 다양한 상품을 판매한다”처럼 저장할 필요가 없는 업무 배경은 엔터티가 아니다. 엔터티가 되려면 관리할 필요가 있고, 같은 종류의 사례가 여러 개 있어야 하며, 이를 설명할 정보 항목이 있어야 한다.

엔터티를 정했다면 각 대상에 무엇을 기록할지 정해야 한다. 고객번호 1001번인 김민수처럼 엔터티에 속하는 개별 사례를 인스턴스라 하고, 고객번호·이름·가입일자처럼 인스턴스를 설명하는 정보 항목을 속성이라 한다. 데이터베이스는 이런 정보를 표 형태로 저장하는데, 표는 테이블, 표의 한 줄은 행, 세로 항목은 컬럼이라 부르며 각각 엔터티·인스턴스·속성에 대응한다. 그런데 같은 속성도 사람마다 다르게 이해할 수 있다. 현업 담당자는 ‘주문일자’를 주문 버튼을 누른 시각으로 보지만 개발자는 결제가 끝난 시각으로 저장할 수 있으므로, 그 속성에 어떤 자료형과 범위의 값만 허용할지도 함께 정해야 한다. 이 값의 규칙을 도메인이라 한다.

관리 대상을 나누어 정했더라도 그 대상들이 서로 어떻게 연결되는지는 별도로 정해야 한다. 고객이 주문을 하고 주문에 결제가 따르는 것처럼 엔터티 사이의 업무상 연결을 관계라 하며, 현업 담당자가 모든 주문에 고객이 있어야 한다고 여겨도 개발자는 비회원 주문을 허용할 수 있으므로 이 연결 규칙도 합의가 필요하다. 회원 한 명은 주문이 없거나 여러 건일 수 있지만, 주문 한 건은 반드시 회원 한 명에게 속한다. 한쪽이 상대방과 최대 몇 개까지 연결되는지는 관계차수, 최소 몇 개가 있어야 하는지는 관계선택성이라 하며, 최소 연결 수가 0이면 선택참여, 1이면 필수참여다. 같은 관계라도 회원에서 주문을 읽을 때와 주문에서 회원을 읽을 때 뜻이 달라지므로 양쪽을 각각 해석해야 한다.

엔터티에는 같은 종류의 인스턴스가 여러 개 있으므로 각 인스턴스를 구별할 기준이 필요하다. 이름만으로 고객을 구별하면 동명이인을 가려낼 수 없지만, 고객마다 다른 고객번호를 부여하면 정확히 구별할 수 있다. 이처럼 인스턴스를 구별하는 기준을 식별자라 하며, 그중 엔터티를 대표하는 것을 주식별자라 한다. 주식별자는 모든 인스턴스를 유일하게 구별할 수 있어야 하고(유일성), 구별에 필요한 최소한의 속성만으로 이루어져야 하며(최소성), 값이 쉽게 바뀌지 않아야 하고(불변성), 비어 있어서는 안 된다(존재성).

합의한 주식별자는 실제 테이블에서 컬럼으로 구현되는데, 이 컬럼이 기본키, 즉 PK이며 고객 테이블에서는 고객번호가 기본키가 된다. 다른 테이블의 기본키를 그대로 가져와 두 테이블의 관계를 나타내는 컬럼은 외래키, 즉 FK라 하는데, 주문 테이블의 고객번호는 고객 테이블의 기본키를 가리키는 외래키다. 외래키에는 부모 테이블에 실제로 존재하는 값만 들어가야 하며, 이 규칙을 참조무결성이라 한다. 고객 테이블에 없는 고객번호가 주문 테이블에 저장되면 그 주문이 누구의 것인지 알 수 없기 때문이다.

엔터티와 관계를 이렇게 나누었더라도, 한 테이블에 서로 다른 사실을 함께 담으면 다른 문제가 생긴다. 학생 수강 정보를 담은 다음 테이블에서 이 문제가 나타난다.

| 학생번호 | 학생이름 | 과목번호 | 과목명    |
|------|------|------|--------|
| 1001 | 김민수  | C01  | 데이터베이스 |
| 1001 | 김민수  | C02  | 통계학    |
| 1002 | 이서연  | C01  | 데이터베이스 |

이 테이블의 한 행은 학생 한 명이 과목 하나를 수강한 기록이며, 학생번호와 과목번호를 함께 봐야 특정 행을 구별할 수 있으므로 이 둘을 합친 기본키를 복합키라 한다. 그런데 학생번호가 같으면 학생이름도 항상 같고, 과목번호가 같으면 과목명도 항

상 같다. 이렇게 한 값이 정해지면 다른 값이 하나로 정해지는 관계를 함수 종속이라 하며, 값을 정하는 쪽을 결정자, 정해지는 쪽을 종속자라 한다.

학생번호가 같은 행마다 학생이름이 반복되므로, 수강생이 없는 새 과목은 등록할 수 없고 이름을 일부 행에서만 고치면 같은 사람의 이름이 기록마다 달라지며 마지막 수강 기록을 지우면 과목 정보까지 함께 사라진다. 이렇게 반복 저장 때문에 삽입·수정·삭제 과정에서 생기는 부작용을 이상현상이라 한다. 함수 종속을 분석해 서로 다른 사실을 별도의 테이블로 나누어 이상현상을 줄이는 작업이 정규화다.

정규화는 단계를 밟아 진행된다. 한 칸에 여러 값을 넣지 않고 원자값 하나만 남기는 단계가 제1정규형이며, 앞의 표를 학생번호·학생이름을 담은 학생 테이블, 과목번호·과목명을 담은 과목 테이블, 학생번호·과목번호만 남긴 수강 테이블로 나누면 학생이름과 과목명이 각각 한 번씩만 저장된다. 복합키 전체가 아니라 일부에만 종속되는 속성, 즉 부분 함수 종속 관계에 있는 속성을 분리하는 단계가 제2정규형이고, 기본키가 아닌 속성을 거쳐 다른 속성이 정해지는 이행 함수 종속을 없애는 단계가 제3정규형이다. 더 엄격한 기준인 BCNF는 어떤 값을 결정하는 결정자가 반드시 슈퍼키, 즉 행을 유일하게 구별할 수 있는 컬럼 조합이어야 한다는 기준이며, 슈퍼키 중 더 줄일 수 없는 최소 조합인 후보키와는 구별된다.

테이블을 나누면 정보를 다시 볼 때 여러 테이블을 함께 봐야 한다. 나눈 테이블을 다시 연결했을 때 원래 정보가 그대로 복원되어야 하며, 이런 분해를 무손실 분해라 한다. 수강 테이블을 학생번호로 학생 테이블과, 과목번호로 과목 테이블과 연결하면 수강 기록에 학생이름과 과목명을 함께 볼 수 있는데, 이렇게 여러 테이블에 나뉜 정보를 공통 값으로 연결해 하나의 조회 결과로 합치는 것이 조인이다. 조인의 목적은 나뉜 정보를 통합해 조회하는 데 있으며 조회 속도를 높이는 기법은 아니다.

테이블이 여러 개로 나뉘면 조회할 때마다 조인이 늘어나 데이터가 많을수록 조회 시간이 길어질 수 있다. 실제로 성능 문제가 확인되었을 때 정규화된 구조를 일부러 되돌리거나 바꾸는 것이 반정규화이며, 자주 함께 조회하는 테이블을 합치거나 자주 필요한 값을 다른 테이블에 복사해 두거나 계산 결과를 미리 저장해 두는 방식으로 이루어진다. 정규화가 중복과 이상현상을 줄이는 것이 목적이라면, 반정규화는 조회 성능을 높이는 것이 목적이다. 값을 여러 곳에 저장하면 한쪽만 고쳤을 때 같은 정보가 서로 다르게 남을 위험이 있으므로, 반정규화는 성능 개선 효과가 그 위험보다 클 때에만 선택해야 한다.

구조를 올바르게 설계해도 운영 중 규칙이 깨지면 데이터는 다시 어긋난다. 각 행을 서로 구별되게 하는 개체무결성, 외래키가 실제 부모 값을 가리키게 하는 참조무결성, 속성 값의 규칙을 지키게 하는 도메인 무결성을 합쳐 무결성이라 한다. 그런데 주문 저장과 결제 저장처럼 여러 변경이 하나의 업무를 이룰 때는 일부만 처리되어서는 안 되는 경우가 있다. 주문은 저장되었는데 결제 저장에 실패하면 결제하지 않은 주문이 접수된 것처럼 남으므로, 여러 작업을 모두 성공시키거나 모두 취소하도록 묶는 규칙이 트랜잭션이다. 모든 작업이 성공하면 결과를 확정하는 COMMIT, 하나라도 실패하면 변경을 되돌리는 ROLLBACK, 특정 지점까지만 되돌리는 SAVEPOINT로 이를 제어한다. 주문이 고객 없이 존재할 수 없다는 요구는 필수참여의 문제이고, 없는 고객번호로 주문을 만들 수 없다는 요구는 참조무결성의 문제이며, 주문 저장과 결제 저장이 모두 성공하거나 실패해야 한다는 요구는 트랜잭션의 문제로, 세 가지는 겹쳐 보이지만 서로 다른 규칙이다.

값이 아예 없거나 아직 알 수 없는 상태도 표현할 방법이 필요하며, 이 상태를 NULL이라 한다. NULL은 숫자 0이나 공백과 다르므로 연락처를 아직 등록하지 않은 고객의 연락처 컬럼에는 NULL이 들어간다.

| 상황                            | 처리 방식                        |
|-------------------------------|------------------------------|
| 컬럼과 NULL 비교                   | 결과는 UNKNOWN이며 조회 조건에서 제외된다   |
| 값 존재 여부 확인                    | IS NULL 또는 IS NOT NULL로 확인한다 |
| NULL이 포함된 산술식                 | 계산 결과도 NULL이 된다              |
| COUNT(*)                      | NULL이 있는 행도 포함해 센다           |
| COUNT(컬럼), SUM, AVG, MAX, MIN | 해당 컬럼의 NULL은 제외하고 계산한다       |
| Oracle의 빈 문자열                 | NULL로 처리되지만 공백과 숫자 0은 예외다    |

NULL은 일반 값처럼 비교할 수 없어 어떤 컬럼이 NULL과 같은지 묻는 비교식의 결과는 참도 거짓도 아닌 UNKNOWN이 되므로, NULL 여부는 IS NULL이나 IS NOT NULL로 확인해야 한다. 이 때문에 A와 B를 더한 뒤 합계를 구하는 계산과 A의 합계와 B의 합계를 각각 구해 더하는 계산은 다른 값을 낼 수 있는데, 앞의 방식은 한 행에서라도 NULL이 있으면 그 행 전체가 합계에서 빠지지만 뒤의 방식은 두 컬럼의 NULL을 각각 제외하고 계산하기 때문이다.

데이터 모델은 업무를 서로 다르게 이해하던 관점들을 하나의 구조로 정리한 결과다. 이후의 데이터베이스 기술은 그 구조가 저장되고 조회되고 바뀌는 과정에서 중복되거나 어긋나거나 일부만 처리되는 문제를 통제하기 위해 쓰인다. 결국 데이터 모델링은 업무 의미에 대한 합의와, 그 합의가 실제 데이터에서 흔들리지 않게 하는 기술적 통제를 잇는 설계 과정이다.